

Задача C1. Робот-курьер

Это задача на реализацию. Основными идейными сложностями являются поиск пути и преодоление светофора за не более чем 25 запросов ожидания.

Для поиска пути можно было использовать `dfs` или `bfs` (поиск в глубину или поиск в ширину). Решения на Python, использовавшие `dfs`, могли столкнуться с трудностями, связанными с неэффективностью рекурсии или с недостаточным стеком рекурсии, поэтому лучше было сразу писать поиск в ширину.

Перечислим, как удобно хранить данные в программе.

- Будем хранить `dir` — текущее направление. В варианте задачи с поворотами типа «rotate» стоило упорядочить все направления против часовой стрелки, чтобы можно было находить угол поворота через разницу их позиций в этой последовательности. Также удобно для каждого направления в `dict` или `map` хранить соответствующие ему перемещения по каждой из двух координат.
- Также заведем словарь/ассоциативный массив (`dict` или `map`) `lights`, сопоставляющий тройкам (r, c, LR) информацию о светофорах, расположенных рядом с переходами в соответствующих клетках. Здесь (r, c) задает позицию перехода, а `LR` — флаг, отвечающий за направление (`true` для направлений «влево» и «вверх», и `false` иначе).

Тогда весь алгоритм заключается в следующем: найдем с помощью поиска в ширину путь из стартовой клетки в конечную по проходимым клеткам, то есть по клеткам тротуара и клеткам, соответствующие которым ключи есть в `lights`, а затем, двигаясь по этому пути, будем убеждаться, что каждый переход безопасен.

Для поиска в ширину достаточно, доставая очередную клетку из очереди, перебирать всех ее соседей (то есть перебирать все четыре направления движения и проверять, что соответствующее направление движения не выводит робота за пределы карты), и для каждой соседней клетки, в которой роботу разрешено находиться, обновлять до нее расстояние.

Затем, чтобы двигаться по найденному пути, будем поддерживать текущее положение робота (r, c) и его направление движения `dir`, и для каждой следующей клетки выполнять следующие действия.

1. Если клетка расположена не в направлении `dir` от текущей, выполним поворот и обновим направление.
2. Если соответствующая клетка есть в `lights`, выполним запрос к камере.
3. Пусть светофор в текущем направлении, то есть `lights[(r, c, dir ∈ {'L', 'U'})]`, задается числами a_i и b_i . Тогда посмотрим на ячейку изображения в строке $h - a_i$ и столбце $\frac{w}{2} + b_i$, и определим цвет светофора.
4. Пока светофор красный, будем ждать какое-то время и повторять запрос к камере.
5. Как только светофор стал зеленым, либо если следующая клетка в принципе была клеткой тротуара, совершим перемещение вперед.

Положение нужного пикселя на изображении могло быть в немного других индексах (на ± 1 отличающихся от указанных выше) в зависимости от вашего варианта задачи и знака числа b_i .

Осталось только понять, как оптимально ждать зеленый сигнал светофора. Ждать по 1 секунде не получится, так как может оказаться, что за 25 секунд светофор не переключится. Но здесь есть две хорошие стратегии, которые позволяют уложиться в ограничение на число запросов: ждать случайное время или ждать каждый раз в два раза больше, чем в предыдущий.

Если ждать случайное время, то с каждым ожиданием будет вероятность 0.5 «попадания» в нужный цвет, что дает вероятность меньше 10^{-6} получить красный хотя бы 20 раз подряд. Ожидание со временами, равными степеням двойки, позволяет гарантировать, что мы не пропустим следующее переключение светофора (так как если предыдущих ожиданий не хватило для переключения, то следующим не получится «перескочить» через целый период).

Задача D1. Скрещивание

Сразу отметим, что второй вариант отличался от первого тем, что вместо чисел явно была дана их битовая запись. Можно было преобразовать битовую запись в число, а можно было просто работать с ней, как со строкой, от этого сложность и время работы решения не менялись.

Теперь перейдем к решению. Пойдем с конца: пусть ответом на задачу является число x . Посмотрим на его битовую запись и найдем в ней первую (старшую) единицу $\text{lead}(x)$. Пусть она стоит на позиции y с конца, то есть соответствует 2^y при разложении x в сумму степеней двойки. Тогда заметим, что любой a_i , у которого $\text{lead}(a_i) > y$, то есть старшая единица стоит раньше, должен быть поставлен в пару с a_j , у которого все биты старше y совпадают с a_i , чтобы их **xor** давал в старших битах 0.

Таким образом, чтобы в ответе lead был равен y , все a_i с $\text{lead}(a_i) > y$ должны разбиваться на пары с одинаковым $\text{head}_{y+1}(a_i) = a_i \gg (y + 1)$. Здесь за $\gg$ обозначен битовый сдвиг вправо, то есть операция, отбрасывающая последние биты числа. В данном случае $a_i \gg (y + 1)$ отбрасывает все биты с нулевого по y -й включительно.

Алгоритм получается следующий: переберем y от 29 до 0 (можно вместо перебора сделать двоичный поиск, чтобы немного увеличить эффективность), сгруппируем все a_i по их head_y , то есть по $30 - y$ старшим битам, и проверим, что все группы, у которых head_y ненулевой, содержат четное число элементов. Если это так, то можно добиться того, чтобы во всех $30 - y$ старших битах у каждого числа в ответе стояли нули, просто сгруппировав исходные числа по парам внутри групп.

Как только мы нашли такой y , для которого хотя бы одна из групп оказалась нечетного размера, понятно, что в ответе в соответствующем бите будет стоять 1. При этом по всем head_{y-1} группы все еще четного размера, поэтому числа надо разбивать на пары внутри таких групп. Тогда сгруппируем a_i по их head_{y-1} и внутри каждой группы выделим те, у которых следующий бит равен 0 и те, у которых следующий бит равен 1.

После этого для каждой группы решим задачу независимо.

- Если количество тех, у которых следующий бит равен 1, четно, то их можно внутри этой группы разбить на пары так, чтобы все результирующие числа имели в следующем бите 0, и в таком случае они все будут меньше ответа.
- Иначе можно разбить на пары все, кроме одного. А еще одно число либо оставить как есть, и тогда ответ для этой группы просто будет равен минимальному из чисел с единицей в следующем бите, либо сопоставить ему в пару число из той же группы, но с нулем в следующем бите.

В таком случае достаточно найти $\min_{\substack{a_i \in \text{group}_0 \\ a_j \in \text{group}_1}} a_i \oplus a_j$, где group_0 и group_1 — элементы текущей

группы с нулем или единицей в следующем бите, соответственно.

Поиск такого минимума является классической задачей, которая решается с помощью битового бора: будем воспринимать каждое число как строку из 30 нулей и единиц, добавим все элементы одной подгруппы в такой бор, а для каждого элемента второй подгруппы будем искать ему пару, минимизирующую их **xor**, стараясь каждый раз в боре идти по совпадающему биту.

Теперь, когда в каждой группе получено минимально возможное максимальное число, ответом будет максимум из ответов для всех групп. Общее время работы такого решения — $\mathcal{O}(n \log A)$, где A — ограничение сверху на значения a_i .